

Match 3 - Retrieval-Augmented Generation

Why this matters

Every organization holds information no public model has ever seen: product specifications, roadmaps, production logs, bug databases, spreadsheets of part numbers. Ask a general-purpose model about any of it and you get a fluent, confident, invented answer. Retrieval-Augmented Generation is how that is fixed — the model answers from documents you give it, and tells you which document each answer came from.

This challenge is graded on exactly those two things: the answer, and the citation. A right answer from the wrong source does not score, because in the enterprise case it is indistinguishable from a lucky guess.

What you are building

A container that is pointed at a folder of mixed documents, then asked questions about them. For each question, it returns a short answer, and the list of files the answer comes from. When the folder does not contain the answer, it says nothing.

You choose the model, the retrieval method, and the document parsers. We do not prescribe an embedding model, a vector database or a framework — only the base image, the way your code is invoked, and the shape of what it returns.

How your code is called

As in Match 2, there is no service to run and no port to expose, and your container must stay running for the whole evaluation. Unlike Match 2 there are TWO invocations, and the difference matters a great deal to your score.

First, once, before any question is asked and before any clock that affects your score is running, we ask you to index the corpus:

```
python3 /app/app.py --index /app/corpus
```

Then, once per question, we ask the question. Each question is a separate process:

```
python3 /app/app.py --corpus /app/corpus \  
  --query-id query_01 \  
  --query "What is the maximum junction temperature of the TQ-40?"
```

Your script writes its answer to `/app/output/<query-id>_output.json` — so the example above writes `/app/output/query_01_output.json`. The query id is given to you; do not invent it, and do not derive it from the question text.

Expected output — `/app/output/query_01_output.json`:

```
{  
  "answer": "94",
```

```
"citations": ["specs/tq40_datasheet_r2.pdf"],  
"confidence": 0.91  
}
```

Field	Type	Required	Notes
answer	string	yes	The value only. Empty string when the corpus does not answer the question.
citations	list of strings	yes	Paths of the files the answer came from. Empty list when there is no answer. Absolute (/app/corpus/specs/x.pdf) or corpus-relative (specs/x.pdf) both accepted.
confidence	number	no	Recorded, not scored.

Both answer and citations must be present on every response, including when you have no answer. A missing field is a malformed response and scores zero for that question — we do not infer a refusal from a missing key, because a crashed writer and a considered refusal would then look identical.

Answers — give the value, and the whole value

Answer with the value only: a number, a part number, a version, a quarter, a code. No sentence, no restatement of the question, no explanation. This is the same convention as Match 2, where the answer is the plate number and not a description of the plate.

Answers are compared after the same normalization used in Match 2 - uppercased, whitespace removed, and the characters - . · _ removed — so 94, 94 C and 94°C all count as the same answer.

Give the COMPLETE value. A qualifier that identifies the value is part of it: a quarter includes its fiscal year (Q3 FY27, not Q3), a revision includes its prefix (REV-C2, not C2). Dropping it makes the answer ambiguous and it is marked wrong.

Citations — read this carefully

This is the part of the challenge most likely to cost you points, so it is worth reading twice.

Your citations are compared as an EXACT SET against the files the answer should have come from. Every required file must be present, and nothing else may be. Returning your whole retrieval — every chunk your search brought back — will fail almost every question, even when your answer is correct.

The test to apply is necessity, not relevance:

- Cite a file only if REMOVING it would make your answer impossible.
- A file that merely discusses the same topic, or that happens to mention the same value somewhere else, is not a source. Citing it is an error.
- Every step of a chain counts. If one file gave you an identifier — a ticket number, an error code, a part number — and a second file gave you the value for that identifier, then removing either would leave you unable to answer, and both must be cited.

Some questions in the graded set are answerable from a single file, and some are not. We do not tell you which is which, and the number of citations you return should follow from the documents you actually used.

When the answer is not there

Some questions cannot be answered from the corpus. For those, the correct response is an empty answer and an empty citation list:

```
{"answer": "", "citations": [], "confidence": 0.0}
```

Guessing is scored as wrong. So is answering from what your model already knows — the corpus describes products that do not exist, so anything your model "remembers" about them it invented.

Return an empty string, not a sentence. "I could not find that" and "I don't know" are prose, and they are scored as wrong answers rather than as refusals.

The corpus you must handle

The folder you are given contains subdirectories, and files of these types. Every one of them holds answers in the graded set, so a pipeline that reads only plain text will lose those questions rather than fail visibly:

Type	What it looks like in the corpus
.pdf	Datasheets and specifications, including multi-page and superseded revisions
.docx	Internal documents such as roadmaps, including text held in tables
.xlsx	Spreadsheets with multiple sheets — part lists, costs, lead times
.csv	Exported tables such as a bug or ticket database
.txt / .log	Production logs and release notes
.py	Source code, where the answer may be a constant or a default value
.png / .jpg	Diagrams and photographs. Some answers appear ONLY in an image, as text printed inside it — you will need a vision model to read them

Four things the folder will do to you

The corpus is deliberately not clean. Your solution must keep working through all four of these, and must not crash, hang, or stop indexing the remaining files:

- An EMPTY directory, with no files in it at all.
- A file of an UNKNOWN type that you have no parser for. Skip it; do not fail.
- A file you have NO PERMISSION to read. Attempting to open it raises; catch it and carry on.
- An ENCRYPTED file that opens but cannot be read without a password you are not given.
Nothing inside it is ever the answer to a graded question.

A corpus walk that raises on the first unreadable file indexes nothing after it. Because the walk order follows the directory listing, which files you end up indexing then depends on filename order — which is how a submission scores well in testing and badly on the graded set.

How you are scored

There are 10 test questions, each worth 20 points. Maximum 200 points.

A question scores only when the answer is correct AND the citations are exactly right. There is no partial credit on a question: a correct answer with an incomplete or over-long citation list scores zero for that question, the same as a wrong answer. Your per-question result tells you which of the two you got wrong.

The hard gates from Section 2 of “All About the Challenges” all still apply — a wrong base image, exceeding the size limit, exceeding a time limit, or using no GPU at all scores zero overall.

Limit	Value	What it covers
Startup	10 minutes	Container start, model load, AND the --index pass over the whole corpus
Per question	30 seconds	One --query invocation, from exec to your JSON file
Whole run	10 minutes	All questions after startup completes
VRAM	1 – 48 GiB	Sampled continuously while you run, not just at rest
Image size	60 GiB	Uncompressed

The indexing pass is charged to the startup budget, which is deliberate and is the one piece of generosity in the timing: it is your opportunity to parse every document once, cheaply, before anything is timed per question. The graded corpus is larger than the sample one below.

Two traps that cost a working solution its score

Both of these have been hit while building the reference solution for this challenge, on the same hardware you are evaluated on.

1. Your script is a new process for every question

We do not call a function in a running program — we execute your script again for each question. A solution that loads its model inside that script loads it ten times, and a solution that indexes the corpus inside that script re-parses every PDF, spreadsheet and image ten times. Either one blows the 30-second budget on every question, and a perfectly good model scores zero.

Keep a long-running process that holds the model and the index, and make /app/app.py a thin client that talks to it — a unix socket is enough. Your container's CMD is the natural place to start it. Alternatively, persist your index to disk during --index and reload only what you need.

2. pip can silently replace ROCm torch with a CUDA build

This one is invisible until runtime and the error message points somewhere else entirely. Installing almost any package that depends on torch lets pip resolve that dependency and install a CUDA build of torch over the ROCm one in the mandated base image. Measured on the evaluation hardware:

```
# before: torch 2.13.0+rocm10.0.0  torchvision 0.28.0+rocm10.0.0
pip install -U transformers
```

```
# after: torch 2.14.0+cu130      torchvision 0.28.0+rocm10.0.0
# RuntimeError: operator torchvision::nms does not exist
```

Nothing mentions torch in that error, and the package you installed is not obviously at fault. Install with `--no-deps`, and verify after building:

```
RUN pip install --no-cache-dir --no-deps -U transformers tokenizers safetensors

docker run --rm your-image:tag python3 -c \
    "import torch; print(torch.__version__)" # must end in +rocm...
```

Check your submission before you send it

These are the same checks our pipeline runs. Add them to the list in Section 2 of “All About the Challenges”:

1. torch is still the ROCm build after every pip install in your Dockerfile.
2. `python3 /app/app.py --index /app/corpus` completes inside the startup budget.
3. A `--query` invocation writes `/app/output/<query-id>_output.json` every time.
4. Your container is still running after all questions — we exec into it repeatedly.
5. Point it at a folder containing an empty subdirectory, a file you have `chmod 000`'d, an encrypted PDF and a file of an unknown type. It must still index everything else and answer.
6. Ask it something the folder does not contain. It must return an empty answer and no citations.

Sample corpus and questions

DOWNLOAD STARTER KIT

A sample corpus is provided so you can see the shape of what you will be given. The graded corpus is different, larger, and harder — but it follows exactly these conventions, and the answer and citation format below is what is graded.

Download `mc3-starter-kit.zip` from the Match 3 page and unpack it. It holds this sample corpus, the ten sample questions below in machine-readable form with their expected answers and citations, a starter container that already implements the contract, and the self-check to run against your image before you submit. Run `setup.sh` first: it recreates the two things a zip file cannot carry — an empty directory, and a file you have no permission to read.

```
unzip mc3-starter-kit.zip
cd mc3-starter-kit && ./setup.sh
```

Two things about that permission case, both of which will otherwise let a submission pass your own testing and fail ours. On Windows it cannot be reproduced at all — `chmod` has no effect there — so test it on Linux, which is what you are evaluated on. And if you test as root, `chmod 000` will not stop you reading the file either, because root bypasses mode bits. At evaluation we drop the `DAC_OVERRIDE` capability, which makes the file genuinely unreadable even for root, so check your own handling with `docker run --cap-drop DAC_OVERRIDE`.

The sample corpus is laid out like this:

```

archive/
  (empty)
engineering/
  ingest_service.py
  meridian_release_notes.txt
logs/
  prod_inference_2026-09-02.log
planning/
  roadmap_fy27.docx
specs/
  backplane_pinout.png
  tq40_datasheet_r1_WITHDRAWN.pdf
  tq40_datasheet_r2.pdf
support/
  asset_label.jpg
  bug_database.csv
  rma_parts.xlsx
vendor/
  internal_audit.txt
  supplier_agreement_ENCRYPTED.pdf
  telemetry_capture.dat

```

The three files under vendor/ are the unreadable, encrypted and unknown-type cases, and archive/ is the empty directory. None of them contains the answer to any question.

These are the sample questions, with the answer and the exact citation expected:

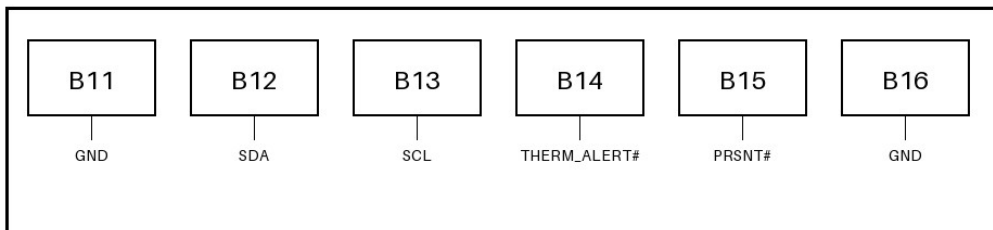
#	Question	Answer	Citations
1	What is the maximum junction temperature of the TQ-40?	94	specs/tq40_datasheet_r2.pdf
2	In which quarter does the TQ-60 enter customer sampling?	Q3 FY27	planning/roadmap_fy27.docx
3	What is the part number of the field-replaceable fan assembly for the TQ-40?	ORR-FAN-2214-B	support/rma_parts.xlsx
4	Which firmware version fixed ticket ORR-1847?	4.3.2	support/bug_database.csv
5	What error code is logged when the thermal throttle engages?	E7731	logs/prod_inference_2026-09-02.log
6	What is the default batch timeout, in seconds, in the ingest service?	180	engineering/ingest_service.py
7	Which backplane pin carries THERM_ALERT# on the TQ-40?	B14	specs/backplane_pinout.png
8	What board revision is printed on the asset label?	REV-C2	support/asset_label.jpg
9	The production log shows a thermal throttle incident. Which firmware release fixed the underlying defect?	4.3.2	logs/prod_inference_2026-09-02.log, support/bug_database.csv
10	What is the unit price of the TQ-40 at 10,000 unit volume?	(empty — not in the corpus)	(none)

Question 10 is the one to study. Its answer does exist in the corpus — it is inside the encrypted file — and it is still scored as unanswerable, because a file you cannot open is not a source. Returning that value means you either guessed it or read something you should not have.

The two questions that need a vision model

Two of the sample questions are answered by text printed inside an image and by nothing else in the corpus. A text-only pipeline cannot score them:

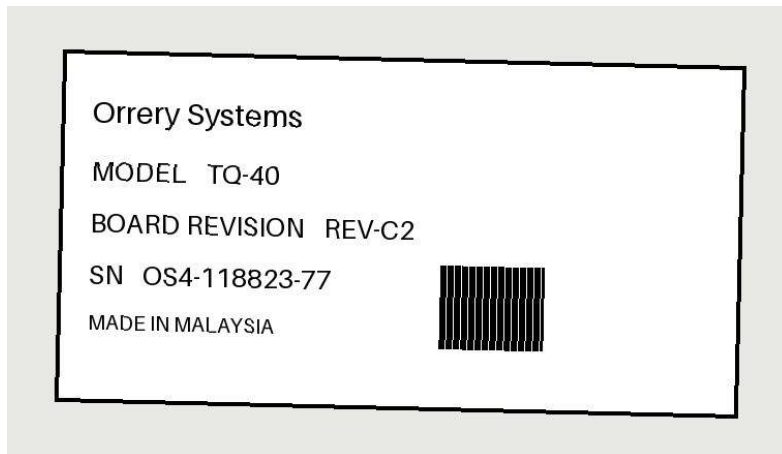
TQ-40 backplane connector - pin assignment



THERM_ALERT# is asserted low when the die exceeds the warning threshold. It is the only open-drain pin in the row.

Sheet 4 of 9 - mechanical drawings are in a separate package.

specs/backplane_pinout.png — the pin carrying THERM_ALERT# appears in no text file



support/asset_label.jpg — the board revision is printed on the label

Useful Links

- OCR with vLLM on ROCm — https://rocm.docs.amd.com/projects/ai-developer-hub/en/latest/notebooks/inference/ocr_vllm.html

- Host Your First LLM on AMD GPU — <https://lablab.ai/ai-tutorials/amd-developer-cloud-host-llm-vllm>
- RAG in production, an overview — <https://lushbinary.com/blog/rag-retrieval-augmented-generation-production-guide/>

We are not recommending a specific retrieval, OCR or vision framework. Several open-source options run well on AMD hardware and evaluating them is part of the work.